1 Problem Statement

A knowledge bot answers questions by searching a knowledge base. Two users can ask the same underlying question in different words:

> "give me all debit transactions"
> "show me all withdrawals"

These mean the same thing, and a good knowledge bot should return the same answer for both. A bot that treats each phrasing as a brand-new, unrelated query can search differently for each one and return different, sometimes contradictory, answers – and there is no way to know this is happening without deliberately testing for it.

At the same time, a real knowledge base is large and constantly growing, which adds three more requirements on top of consistency:

- The bot must handle questions on topics it was never told about in advance, and honestly say "I don't know" rather than forcing a guess – it cannot rely on a hand-written list of every possible question category.
- The bot must serve many tenants/customers whose data must never leak into each other's answers.
- The bot must not repeat expensive search and reasoning work for every repeated or near-duplicate question.

The problem this document addresses: given a large, growing knowledge base, how do we build a retrieval system that (1) treats semantically equivalent queries consistently, (2) scales to millions of documents without paying the cost of an expensive model reading every document, (3) honestly reports when it has no relevant answer, and (4) does all of this without hand-maintaining a closed list of every possible question category up front?

2 Approach

The architecture was arrived at incrementally, each step fixing a limitation of the one before it:

1. **Start from meaning, not keywords.** Use vector embeddings so that queries with similar meaning are recognized as similar, instead of a hand-typed synonym dictionary that only covers phrasings someone thought to write down in advance.
2. **Do not force every query into a pre-declared category.** Search the knowledge base directly instead of routing to a fixed list of intents. This is what makes genuinely new topics answerable and makes "I don't know" a legitimate outcome instead of a forced guess.
3. **Separate "find candidates fast" from "judge them carefully."** Use cheap approximate retrieval (dense ANN + sparse BM25) to narrow a huge corpus down to a small set of candidates, then apply a slower, more accurate reranker only to that small set.
4. **Collapse repeated or near-duplicate questions before they reach retrieval.** Analyze and canonicalize each query using schema values read from the data itself and a self-growing cluster registry, rather than a hand-typed alias list, and cache canonical answers so repeat questions do not re-run the full pipeline.

5. **Validate constantly, and do not claim more than the mechanism actually guarantees.** Every consistency, caching, recall, and tenant-isolation claim below is something that was tested and measured, not assumed. In particular, the query-clustering step in Section 3.11 is *not* guaranteed to group every semantically-equivalent pair of questions – it is order-dependent by construction – and this document does not claim otherwise.

The result of this process is the architecture described below.

3 Solution: Open-World Knowledge Retrieval Architecture

The system is designed to answer questions over a large and continuously changing collection of documents.

The main idea is simple:

> **Do not ask an expensive AI model to read one million documents.** First use fast retrieval methods to find a small set of potentially relevant documents. Then use a more intelligent reranker to carefully judge only those candidates.

For example, suppose the knowledge base contains

$$N = 1{,}000{,}000 \tag{1}$$

documents.

A typical query does *not* compare the query carefully against all one million documents. Instead, the system progressively reduces the search space:

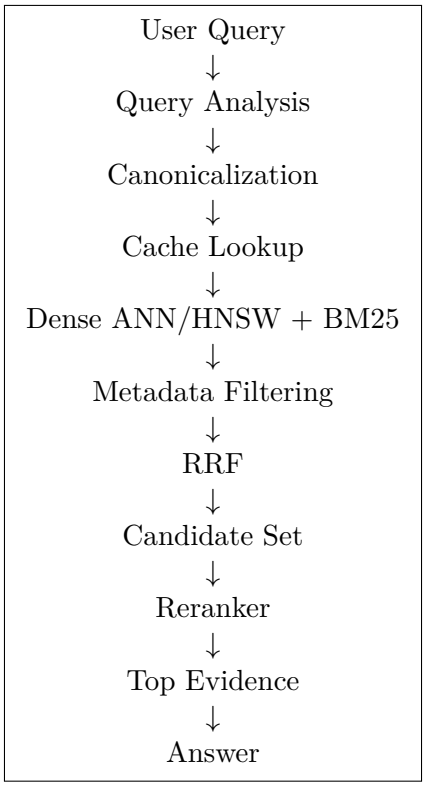
$$\boxed{1{,}000{,}000 \rightarrow 100 \rightarrow \text{fewer strong candidates} \rightarrow \text{top evidence} \rightarrow \text{answer}} \tag{2}$$

The purpose of each stage is therefore different:

- **ANN/HNSW**: find potentially relevant documents quickly.
- **BM25**: find documents using lexical and keyword matches.
- **RRF**: combine the different retrieval rankings.
- **Metadata filtering**: remove documents that do not satisfy required constraints.
- **Reranker**: carefully judge the remaining candidates.
- **Answer generation**: use the strongest evidence to answer the user's question.

3.1 The Big Picture

The complete query-time workflow is:



(3)

The important point is that the system becomes more expensive only after the number of documents has already been reduced substantially.

3.2 What Happens When There Are One Million Documents?

Consider a knowledge base containing one million documents:

$$\text{Knowledge Base} = 1{,}000{,}000 \text{ documents.} \tag{4}$$

Suppose a user asks:

> "Show me debit transactions over $100 from the last 30 days."

The system does not send all one million documents to an AI model.
Instead, the query passes through several stages.

3.2.1 Step 1: Understand the Query

The system first analyzes the query.

It identifies the general meaning of the question and extracts structured information where possible.

For example:

Query information	**Detected value**
Transaction type	DEBIT
Amount	greater than $100
Time period	last 30 days

The query is also converted into an embedding vector for semantic retrieval.
At this point the system has two useful representations:

1. the original natural-language query; and
2. structured constraints extracted from the query.

3.2.2 Step 2: Check the Cache

Before performing retrieval, the system checks whether it has already answered an equivalent canonical query for the same tenant and retrieval constraints.
If there is a valid cache entry:

$$\boxed{\text{Cache Hit} \Rightarrow \text{Return Stored Result}} \tag{5}$$

The expensive retrieval and reranking stages are skipped.
If there is no cache entry:

$$\boxed{\text{Cache Miss} \Rightarrow \text{Run Retrieval}} \tag{6}$$

3.3 The Retrieval Stage

This is where the one-million-document problem is reduced to a manageable number of candidates.
The system uses two complementary retrieval methods:

1. dense ANN/HNSW retrieval; and
2. sparse BM25 retrieval.

3.3.1 Step 3: Dense Retrieval with HNSW

The query embedding is searched against the HNSW index.
Conceptually:

$$\text{1,000,000 documents} \xrightarrow{\text{HNSW}} \text{100 candidate documents.} \tag{7}$$

The exact number is configurable. The example uses 100 simply to make the workflow easy to understand.
The key idea is:

> HNSW is the fast "find me documents that look semantically similar" stage.

It is *not* the final judge of relevance.
The 100 documents returned by HNSW should therefore be thought of as:

> **100 documents that are worth looking at more closely.**

They are not necessarily the 100 documents that actually answer the question.

3.3.2 Step 4: BM25 Retrieval

At the same time, BM25 performs lexical retrieval.

BM25 is particularly useful when the query contains words, names, identifiers, numbers, or terminology that should match the document text.

For example, BM25 may return another group of documents:

$$\text{1,000,000 documents} \xrightarrow{\text{BM25}} \text{100 candidate documents.} \tag{8}$$

The dense and sparse searches therefore provide two different views of the same query.

Method	Strength	Typical failure
HNSW/ANN	Semantic similarity	Can miss exact terminology
BM25	Exact lexical matches	Can miss semantic paraphrases

3.4 What Happens to the 100 + 100 Documents?

This is the important part.

Suppose HNSW returns:

$$H = 100 \text{ documents} \tag{9}$$

and BM25 returns:

$$B = 100 \text{ documents.} \tag{10}$$

The system does *not* automatically have 200 unique documents.

Some documents may appear in both lists.

For example:

$$|H \cup B| \leq 200. \tag{11}$$

If 30 documents occur in both lists, there are only 170 unique candidates.

These rankings are then combined using Reciprocal Rank Fusion (RRF).

3.5 Step 5: Reciprocal Rank Fusion

RRF combines the rankings produced by HNSW and BM25.

The important idea is simple:

> A document that ranks highly in multiple retrieval methods becomes a stronger candidate.

For example:

Document	HNSW Rank	BM25 Rank
Document A	3	7
Document B	2	–
Document C	25	4
Document D	–	8

Document A is interesting because both retrieval methods independently consider it highly relevant.

RRF therefore creates one combined ranking:

$$\text{HNSW Ranking} + \text{BM25 Ranking} \rightarrow \text{Fused Ranking}. \tag{12}$$

The exact RRF score is

$$\text{RRF}(d) = \sum_{r} \frac{1}{k_{\text{RRF}} + \text{rank}_r(d)}, \tag{13}$$

where r represents each retrieval ranking.

The current implementation uses

$$k_{\text{RRF}} = 60. \tag{14}$$

The mathematical formula is less important operationally than the main idea:

RRF turns multiple retrieval rankings into one candidate ranking.

3.6 Step 6: Metadata Filtering

The query may also contain constraints that are not adequately captured by semantic similarity alone.

For example:

"debit transactions over $100 from the last 30 days for tenant ACME"

may require:

Field	**Required value**
Tenant	ACME
Transaction type	DEBIT
Amount	$> \$100$
Date	within last 30 days

A document that does not satisfy these requirements should not become evidence merely because its text is semantically similar.

Therefore:

$$\text{Retrieved Candidates} \rightarrow \text{Metadata Constraints} \rightarrow \text{Eligible Candidates}. \tag{15}$$

This is particularly important for tenant isolation.

A document belonging to another tenant must never be returned simply because its text happens to be similar to the query.

3.7 Step 7: The Candidate Set

After retrieval, fusion, and filtering, the system has a much smaller candidate set.

For example:

$$1{,}000{,}000 \rightarrow 100 \text{ HNSW} + 100 \text{ BM25} \rightarrow \text{fused candidates} \rightarrow 20 \text{ candidates}. \tag{16}$$

The number 20 is only an example. The actual value is configurable.

The important reduction is:

$$1{,}000{,}000 \rightarrow \text{tens of candidates}. \tag{17}$$

This is where the reranker becomes practical.

It would be expensive to ask a large language model to carefully judge one million documents.

It is much more practical to ask it to carefully judge 10–50 candidates.

3.8 Step 8: The Reranker

The reranker is the "take a closer look" stage.

This distinction is important.

The retrieval system mainly asks:

> "Which documents look like they might be relevant?"

The reranker asks:

> "Given this exact question and this exact document, does this document actually answer the question?"

The query and candidate document are provided to the reranking model together:

$$(q, d_i) \rightarrow g(q, d_i) \rightarrow \text{relevance score}. \tag{18}$$

For example, if there are 20 candidates:

$$20 \text{ candidates} \rightarrow \text{reranker} \rightarrow \text{ranked candidates}. \tag{19}$$

The reranker can then identify the strongest evidence.

This is different from simply calculating:

$$\text{embedding similarity} + \text{keyword overlap}. \tag{20}$$

A real reranker can consider the relationship between the query and the candidate passage directly.

3.9 What the Reranker Actually Does

Suppose the user asks:

> "Which debit transactions over $100 occurred in the last 30 days?"

HNSW may retrieve a passage saying:

"The customer frequently makes debit purchases at grocery stores."

This passage is semantically related to the question.
However, it may not contain the actual transactions being requested.
Another candidate may say:

"On July 14, a debit transaction of $127.40 was recorded."

The second passage is much more useful as evidence.
A reranker is designed to distinguish these cases.
Therefore:

$$\text{Semantic Similarity} \neq \text{Actual Answer Relevance.} \tag{21}$$

This distinction is one of the main reasons the reranking stage exists.

3.10 Step 9: Top Evidence

The reranker produces a final ordering:

$$d_7 > d_{12} > d_3 > d_{19} > \cdots \tag{22}$$

The system then retains only the strongest passages.
For example:

$$20 \text{ candidates} \rightarrow 5 \text{ top evidence passages.} \tag{23}$$

The answer-generation model receives these passages rather than the entire one-million-document knowledge base.
Therefore:

$$\boxed{1{,}000{,}000 \rightarrow 100 \rightarrow 20 \rightarrow 5 \rightarrow \text{Answer}} \tag{24}$$

This progressive reduction is the central idea of the architecture.

3.11 Complete Query Workflow

The complete workflow can be summarized as follows.

1. **User asks a question.**

   The system receives the natural-language query.

2. **Analyze the query.**

   The system creates an embedding and extracts structured constraints such as transaction type, amount, date range, and tenant.

3. **Canonicalize the query.**

   Similar queries can be grouped under a shared canonical representation. This grouping is data-derived (schema values read from the documents, plus a self-growing cluster registry) rather than a hand-typed alias list, but it is not a mathematical guarantee: which cluster a query joins, if any, can depend on the order in which queries arrive.

4. **Check the cache.**

   If the same effective retrieval request has already been processed, the cached result can be returned.

5. **Run HNSW.**

   HNSW quickly retrieves a relatively small number of semantically similar documents from the large corpus.

6. **Run BM25.**

   BM25 independently retrieves documents with strong lexical matches.

7. **Apply metadata constraints.**

   Documents that violate required tenant or structured constraints are removed.

8. **Fuse the rankings.**

   RRF combines the dense and sparse rankings.

9. **Create the candidate set.**

   Only a small number of documents continue to the expensive stage.

10. **Rerank the candidates.**

    The external relevance model examines the query and each candidate together.

11. **Select top evidence.**

    The strongest passages are retained.

12. **Generate the answer.**

    The answer-generation stage uses the selected evidence.

3.12 End-to-End Workflow Diagram

Figure 1 shows the same process visually.

3.13 Document Ingestion Workflow

There is also a separate workflow for adding documents to the knowledge base.

A new document follows this path:

$$\boxed{\text{New Document} \rightarrow \text{Embedding} \rightarrow \text{HNSW} \rightarrow \text{Document Store} \rightarrow \text{BM25 Update}} \tag{25}$$

The dense HNSW index supports incremental insertion.
Therefore, a new document can be added without rebuilding the entire dense index.
The current BM25 implementation is different.
If `rank_bm25` is used, adding a document marks the sparse index as dirty:

$$\texttt{bm25\_dirty} = \text{True}. \tag{26}$$

The next BM25 search rebuilds the sparse structure over the current corpus.
Therefore, the current system provides:

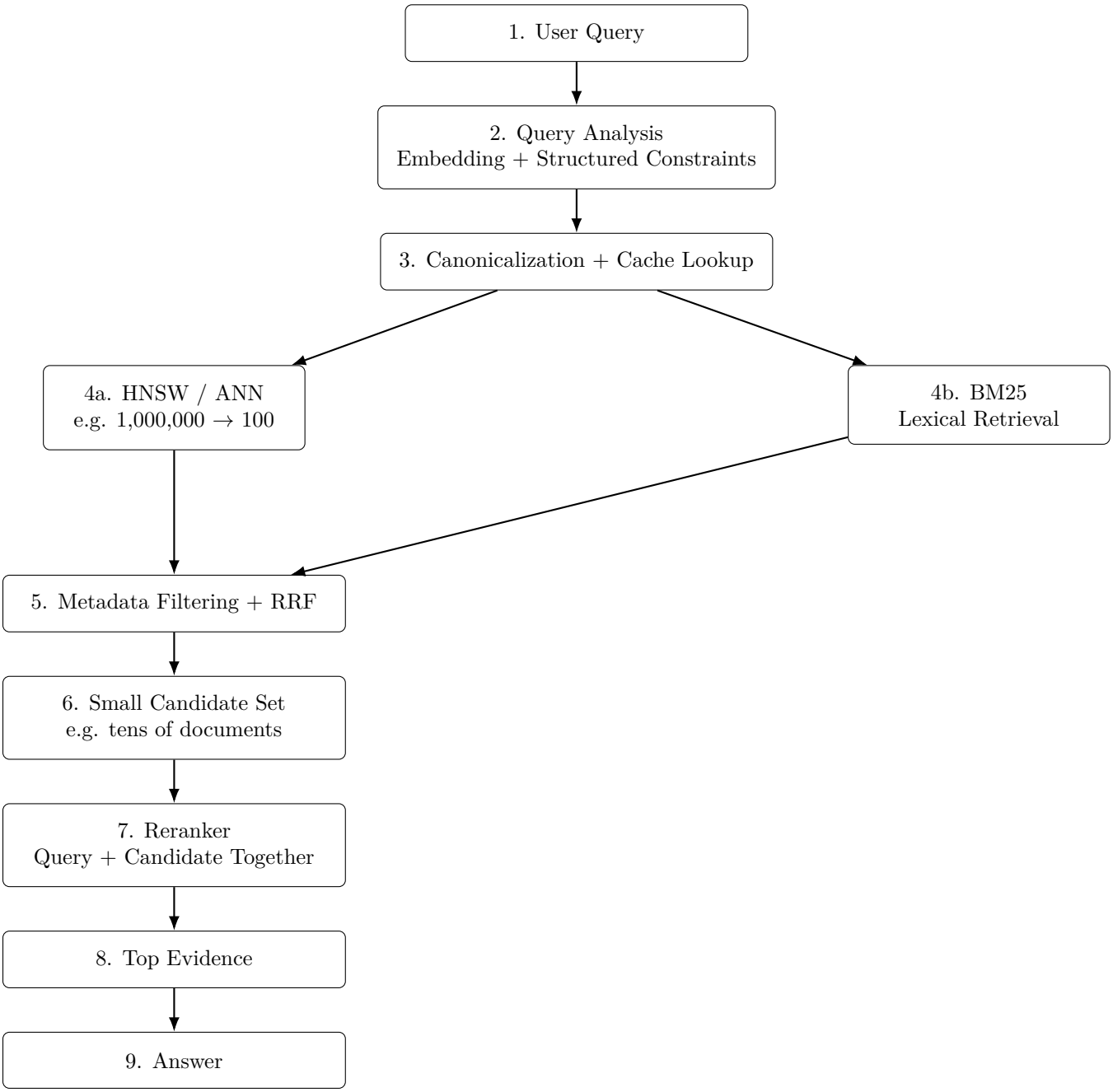


Figure 1: Simplified query workflow. The large document collection is reduced using fast retrieval methods before the expensive reranking stage. HNSW provides semantic candidate retrieval, BM25 provides lexical candidate retrieval, RRF combines the rankings, and the reranker carefully evaluates only the resulting small candidate set.

incremental HNSW insertion + deferred BM25 rebuilding.

It does *not* provide true incremental BM25 indexing.

3.14 Why the Architecture Uses Multiple Stages

Each stage solves a different problem.

Stage	Main purpose
HNSW / ANN	Quickly find semantically similar documents from a very large corpus.
BM25	Find documents containing important query terms and exact terminology.
Metadata filtering	Enforce structured requirements such as tenant, type, amount, or date.
RRF	Combine the different retrieval rankings.
Reranker	Carefully determine which candidates are actually relevant to the question.
Top evidence	Reduce the reranked candidates to the strongest passages.
Answer generation	Use the selected evidence to construct the final response.

3.15 The Most Important Distinction

The architecture should be understood as a sequence of increasingly careful decisions.

$$\begin{aligned} \text{ANN/BM25} &: \text{“Could this document be relevant?”} \\ \text{RRF} &: \text{“How strong is its retrieval position across methods?”} \\ \text{Reranker} &: \text{“Does this document actually answer the question?”} \\ \text{Answer} &: \text{“What should I tell the user based on the evidence?”} \end{aligned} \tag{27}$$

This separation is important.
HNSW is not supposed to perfectly answer the user's question.
Its job is to reduce the search space.
The reranker is not supposed to search one million documents.
Its job is to carefully judge the small set produced by retrieval.
The answer-generation model is not supposed to search the entire database.
Its job is to use the strongest retrieved evidence to formulate the answer.

3.16 Reference Implementation Versus Production System

The current implementation contains several components that are useful for experimentation but may need to be replaced or extended for a large production deployment.

Component	Current approach
Dense retrieval	FAISS HNSW with incremental vector insertion.
Sparse retrieval	`rank_bm25` with deferred full-corpus rebuilding.
Canonical cache	In-memory cache.
Query clustering	In-memory dynamic cluster registry.
Metadata filtering	Application-level filtering in the reference implementation.
Reranking	External relevance model.
ANN evaluation	Optional brute-force exact retrieval for recall measurement.

3.17 Brute-Force Retrieval Is Not the Production Path

The system may also perform brute-force vector search when evaluating the quality of HNSW.

This is useful because it provides an exact reference.

For example:

$$\text{Brute Force} \rightarrow \text{Exact Top-}k \tag{28}$$

can be compared with:

$$\text{HNSW} \rightarrow \text{Approximate Top-}k. \tag{29}$$

The overlap between the two sets measures ANN recall.

However, brute-force search requires comparing the query with the entire corpus.

For one million documents, that means approximately one million similarity comparisons for one query.

Therefore:

$$\boxed{\text{Brute Force} = \text{evaluation}} \tag{30}$$

whereas

$$\boxed{\text{HNSW} = \text{production retrieval}} \tag{31}$$

in the intended architecture.

3.18 Reranking With an External Model

The external reranker is intentionally placed after retrieval.

For example, if retrieval produces 20 candidates:

$$20 \text{ candidates} \rightarrow \text{external reranker} \rightarrow 5 \text{ strong evidence passages.} \tag{32}$$

The external reranker can be implemented using a model such as Claude, Cohere, or another relevance model supported by the application.

The important architectural property is not the specific provider.

The important property is:

The expensive relevance model sees only the small candidate set, not the entire corpus.

This makes a high-quality relevance model practical even when the underlying knowledge base is very large.

3.19 Operational Considerations

The reranker introduces an external dependency.

A production deployment therefore needs to consider:

- authentication,
- network latency,
- API availability,
- rate limits,
- request failures,
- timeouts,
- retry behavior,
- model cost, and
- fallback behavior.

If the reranker is unavailable, the system should have an explicitly defined behavior rather than silently assuming that the fallback ranking is equivalent to the external relevance judgment.

3.20 Caching

Caching can prevent repeated queries from running through the entire pipeline.

The basic behavior is:

$$\text{Query} \rightarrow \text{Canonical Key} \rightarrow \text{Cache}. \tag{33}$$

On a cache hit:

$$\boxed{\text{Cache Hit} \rightarrow \text{Return Previous Result}} \tag{34}$$

On a cache miss:

$$\boxed{\text{Cache Miss} \rightarrow \text{Retrieve} \rightarrow \text{Rerank} \rightarrow \text{Store Result}} \tag{35}$$

The cache must include information that affects the retrieval result, especially tenant identity and effective metadata constraints.

Otherwise, a result generated for one tenant could incorrectly be returned to another tenant.

3.21 Final End-to-End Example

The entire system can be understood through one concrete example.

Suppose the knowledge base contains:

$$\text{1,000,000 documents.} \tag{36}$$

The user asks:

"Show me debit transactions over $100 from the last 30 days."

The system performs the following steps:

1. The query is converted into an embedding.
2. Structured constraints are extracted: `DEBIT`, amount $>$ \$100, and last 30 days.
3. The canonical representation is determined.
4. The cache is checked.
5. On a cache miss, HNSW searches the one-million-document vector index and returns approximately 100 candidates.
6. BM25 independently searches the corpus and returns another group of candidates.
7. Metadata constraints remove documents that do not satisfy the required conditions.
8. RRF combines the remaining dense and sparse rankings.
9. A small candidate set is created, for example 20 documents.
10. The reranker receives the original question and each of the 20 candidates.
11. The reranker scores the candidates according to actual query–document relevance.
12. The strongest passages are selected, for example the top 5.
13. The answer-generation stage receives those passages and constructs the final answer.

The entire process can therefore be summarized as:

$$\boxed{\begin{array}{c}\text{1,000,000 documents}\\ \downarrow\\ \text{HNSW + BM25}\\ \downarrow\\ \text{hundreds of candidates}\\ \downarrow\\ \text{Metadata Filtering + RRF}\\ \downarrow\\ \text{tens of candidates}\\ \downarrow\\ \text{Reranker}\\ \downarrow\\ \text{few strong evidence passages}\\ \downarrow\\ \text{Answer}\end{array}} \tag{37}$$

3.22 Key Takeaway

The most important idea in the architecture is that retrieval and reranking have different jobs.

Retrieval is about speed and recall.

Reranking is about precision.

With one million documents, HNSW does not need to identify the perfect answer immediately. It only needs to reduce one million documents to a manageable set of good possibilities.

BM25 provides a second retrieval signal so that important lexical matches are not lost.

RRF combines those signals.

The reranker then examines the small candidate set carefully and determines which passages are actually useful for answering the question.

Therefore, the architecture is:

$$\boxed{\text{Large Corpus} \rightarrow \text{Fast Retrieval} \rightarrow \text{Small Candidate Set} \rightarrow \text{Careful Reranking} \rightarrow \text{Evidence} \rightarrow \text{Answer}} \tag{38}$$

This separation allows the system to scale to a large document collection without requiring an expensive reasoning model to inspect the entire corpus for every query.

4 What Is Needed for Production

The architecture in Section 3 is correct and has been validated at small scale. Several of its components, however, are reference implementations chosen to make the design testable, not components ready to carry real production traffic. This section states plainly what changes before this system should serve real users.

4.1 Embedding Model

The reference implementation uses a simple word-vector average as its embedding. This is the single biggest limiting factor on answer quality: it struggles to separate on-topic from off-topic queries and under-serves precision throughout the pipeline.

Required: replace it with a modern, contrastively-trained sentence-embedding model (e.g. a hosted embeddings API, or a properly downloaded sentence-transformer model). This is a single function boundary in the code, but it is the highest-leverage change available – most of the precision problems observed in testing trace back to this one dependency, not to the surrounding logic.

4.2 Vector Search Infrastructure

The reference implementation runs FAISS HNSW in-process, in memory, with no persistence, replication, or sharding.

Required: a managed or self-hosted vector database (e.g. Pinecone, Weaviate, Qdrant, Milvus, or pgvector/OpenSearch with a vector-search extension) that provides persistence, horizontal scaling across shards, native metadata-filtered search, and recovery after a restart. Recall@k against a brute-force baseline (Section 3.17) should be re-measured against the production index and monitored on an ongoing schedule, not verified once at launch.

4.3 Sparse / Lexical Search

`rank_bm25` has no incremental-update API: every rebuild recomputes statistics over the entire corpus. The reference implementation defers and batches these rebuilds, which helps, but does not make the underlying rebuild cost disappear.

Required: a real inverted-index search engine (Elasticsearch, OpenSearch, or a Lucene-based service) that supports genuine incremental indexing. This is a hard requirement at large corpus sizes, not an optimization.

4.4 Reranker

The reference implementation's default reranker is a heuristic blend of embedding similarity and keyword overlap – explicitly a stand-in, not a production relevance judge.

Required: a real cross-encoder or LLM-based reranker that scores the query and each candidate document jointly. This requires real, securely-managed credentials (see Section 3.19), a defined timeout and retry policy, and – critically – an explicit, logged fallback path if the reranker call fails, rather than silently returning heuristic results as if they were the real judgment. Any reranker backend chosen should be verified against its provider's current service status before being set as a default; hosted inference services can be deprecated or retired with limited notice.

4.5 Canonicalization / Query Clustering

The auto-growing cluster registry in Section 3.11 is single-pass, nearest-centroid clustering: it is order-dependent by construction, and two semantically-equivalent queries are not guaranteed to join the same cluster.

Required, depending on the actual consistency requirement:

- If near-consistency is acceptable, run the registry as-is but monitor cluster counts and join rates in production, and periodically audit a sample of queries that failed to join an expected cluster.
- If a stronger guarantee is required, add a periodic offline re-clustering pass over accumulated query traffic (removing the order-dependence within each batch), or bound canonicalization to a well-defined schema/vocabulary where deterministic exact-match resolution is achievable.

Either way, the registry and cache must remain scoped by tenant, as implemented and tested in Section 3.6 – this is not optional and must be re-verified after any change to the canonicalization logic.

4.6 Caching

The reference cache is in-memory and process-local: it does not survive a restart and is not shared across multiple server instances.

Required: a shared, external cache (e.g. Redis or a managed key-value store) with an explicit eviction policy (LRU/TTL) and, quite importantly, **invalidation when the underlying source documents change or are deleted** – a cached answer that cites a document which has since been updated or removed must not continue to be served.

4.7 Continuous Validation and Monitoring

Ad hoc testing on a handful of paraphrase clusters, as used during development, does not scale to production traffic.

Required:

- A curated, versioned set of paraphrase and control (adversarial, off-topic) query clusters, run as an automated regression suite in CI on every model, index, or reranker-configuration change.
- Production sampling: log a slice of live queries, their retrieved documents, and confidence scores for periodic audit.
- An explicit "no relevant result" rate tracked over time – a sudden shift signals either a genuine change in what users are asking or a broken component upstream.
- Shadow/canary deployment for embedding-model or index changes: run the new configuration against real traffic in parallel, compare against production, before cutover.

4.8 Security and Data Governance

- Structured filters derived from query analysis must be applied through parameterized queries against the underlying store, never string-concatenated, regardless of how well-behaved the canonicalization layer appears to be.
- Tenant isolation (Section 3.6) must be enforced at every layer that touches tenant data, including the cache – a cache keyed only on query similarity, without a tenant check, is a data-leak path.
- Data retention and deletion requirements for cached answers and logged queries should follow the same policy as the underlying documents they were derived from.

4.9 Production Readiness Checklist

Component	Status before production
Embedding model	Must be replaced with a real sentence-embedding model.
Vector index	Must move to a persistent, shardable vector database.
Sparse index	Must move to a search engine with true incremental indexing.
Reranker	Must be a real cross-encoder/LLM call with a verified, monitored provider and a defined fallback.
Canonicalization	Must be monitored for join/cluster drift, or bounded to a deterministic schema if strict guarantees are required.
Cache	Must move to a shared external store with eviction and invalidation.
Validation	Must become an automated, versioned regression suite plus production sampling.
Tenant isolation	Already implemented and tested; must be re-verified after any change to caching or canonicalization.
Security	Parameterized queries and enforced tenant scoping at every layer.

5 Future Extensions

The architecture in Section 3 was implemented and validated against a document-only knowledge base. This section describes natural extensions beyond that scope. **These extensions are pro-**

posed, not implemented or tested – they are included here as a forward-looking design target, following the same standard the rest of this document holds itself to: nothing above this line is claimed to work unless it was actually tested, and the same discipline applies going forward – these should be validated with the same rigor (paraphrase clusters, control clusters, recall measurement) before being trusted.

Three extensions are described: a structured retrieval path over a knowledge graph, an explicit context-optimization stage, and an explicit LLM-based answer-generation stage that consumes both text and graph evidence.

5.1 Structured Retrieval via a Knowledge Graph

The current architecture retrieves unstructured text passages (dense) and lexical matches (sparse). Many questions, however, are naturally relational – "which accounts are linked to this one," "what entities are connected to this transaction" – and are better answered by traversing explicit relationships than by searching text.

A knowledge-graph retrieval path would run alongside dense and sparse retrieval, not replace them:

Retrieval path	**Answers questions about**
Semantic (ANN)	What does this sound like semantically
Lexical (BM25)	What contains these exact terms
Structured (Knowledge Graph)	What is connected to what

The output of this path is not a ranked list of passages but a set of graph facts (entities and relationships) relevant to the query, which is fused with the text-retrieval results downstream.

5.2 Context Optimization

After reranking, the current architecture passes the top evidence directly to the answer stage. As the number and variety of evidence sources grows (multiple text passages plus graph facts), a dedicated context-optimization stage becomes necessary: selecting, deduplicating, and ordering evidence to fit within a model's context budget, rather than concatenating everything the reranker returned.

This stage would be responsible for:

- removing redundant or overlapping evidence,
- ordering evidence by relevance so the most important facts are not truncated if the context budget is exceeded, and
- merging text evidence and graph facts into a single evidence package the answer stage can consume.

5.3 Explicit LLM-Based Answer Generation

The current architecture ends at "top evidence" and treats answer generation as an external concern. Making this stage explicit means an LLM receives the optimized evidence package (text evidence and graph facts together) and produces the final natural-language answer, rather than the evidence being handed to the user directly.

This is a materially different guarantee than everything in Section 3: retrieval consistency (same evidence for equivalent queries) does not by itself guarantee answer consistency, because generation is a separate, typically non-deterministic step. If this stage is added, it needs its own validation layer – checking that the generated answer is actually grounded in the supplied evidence (faithfulness), not just that the evidence was consistent going in.

5.4 Extended Workflow Diagram

Figure 2 shows the extended workflow described above, layered on top of the architecture in Section 3.

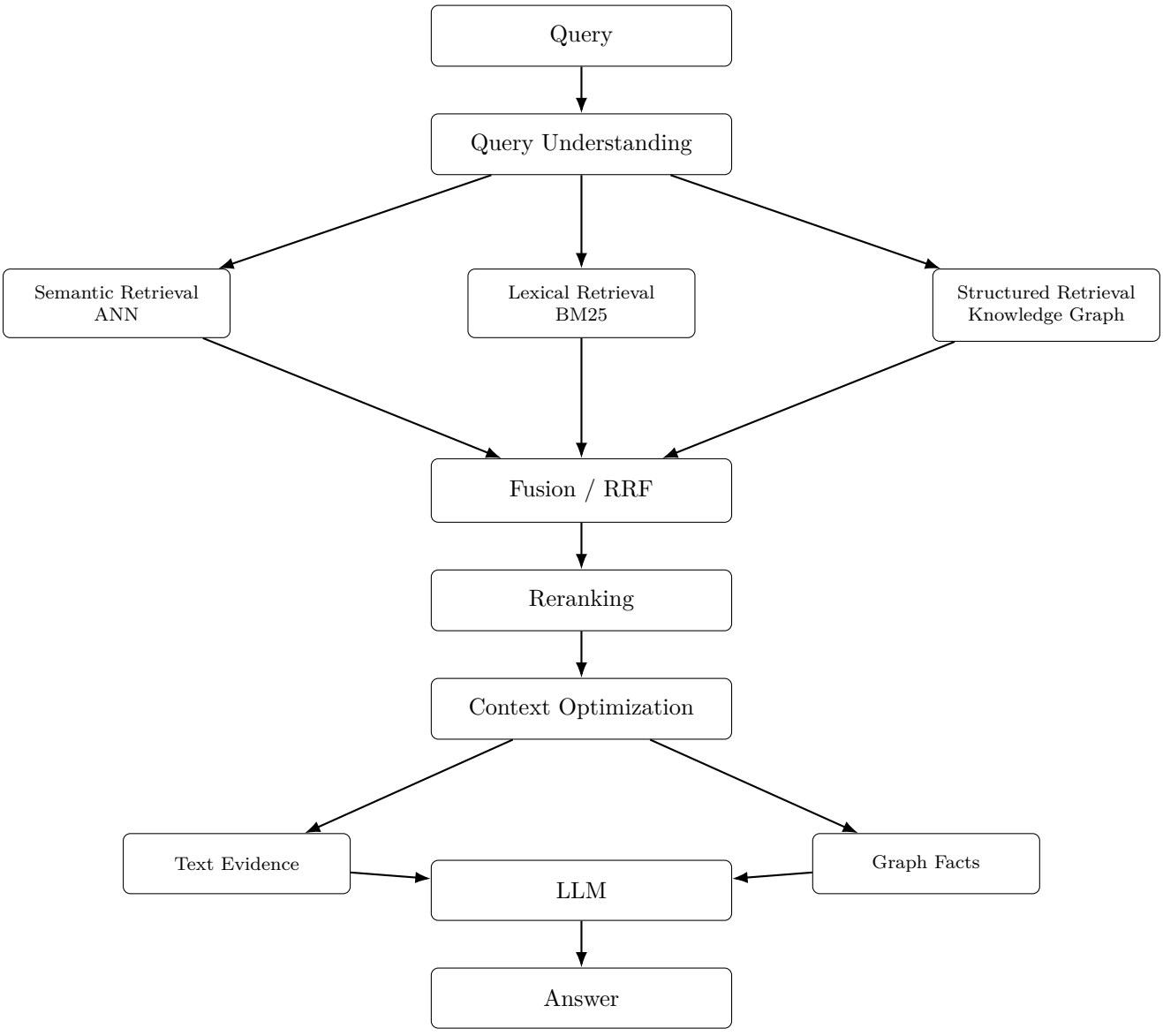


Figure 2: Proposed extended workflow. New components relative to Figure 1 are the structured (knowledge-graph) retrieval path, the context-optimization stage, and the explicit LLM answer-generation stage consuming both text evidence and graph facts. None of these are implemented or validated in the current system.